arXiv:2411.19542v1 [cs.DC] 29 Nov 2024

A dynamic parallel method for performance optimization on hybrid CPUs

Luo Yu[1*] **Liu Yucheng**[1*] **Shen Haihao**[1]
[1]Intel Corporation
{yu.luo,yucheng.liu,haihao.shen}@intel.com

Abstract

The AIPC concept is gaining popularity, and more and more hybrid CPUs will be running AI models on client devices. However, the current AI inference framework overlooks the imbalanced hardware capability of hybrid CPUs, leading to low inference performance. To address this issue, we have introduced a dynamic parallel method for hybrid CPUs, which significantly increases LLM inference performance by balancing the workload for each core of a hybrid CPU before the parallel work starts. This method has enabled Neural Speed to achieve more than 90% (on average) of memory bandwidth on two hybrid Intel CPUs.

1 Introduction

Many CPU companies are now adopting a hybrid CPU architecture to strike a balance between performance and energy consumption. A hybrid CPU typically has more than one microarchitecture or multiple working frequencies among its cores. Both types of hybrid CPUs share a common feature - an imbalance in hardware capabilities among the cores. For instance, the Intel Ultra 100 Series and Ryzen AI 300 Series have two microarchitectures [12], [2]. while the Qualcomm Snapdragon X Elite operates with two frequency levels [15]. These new CPUs are specifically designed to handle large AI models, especially LLM models.

Recent LLM quantization work shows that 4-bit weight-only quantization is both accurate and efficient for LLM models [10], [6], [7], [5]. The research also shows that the token generation speed is related to the device's memory bandwidth [17]. It allows many client CPUs to run some LLM models faster than a human reading speed of about 200ms per token [4]. It's also valuable to continue optimizing the performance of a CPU since it shares the same system bandwidth as its high-performance NPU and GPU.

There are many great works about LLM inference optimization on CPUs. Many of them aim at the server CPUs that are not hybrid [14] [9]. $llama.cpp$ [8] is a popular low-bit LLM inference framework with good performance on many client CPUs, including hybrid CPUs. Nonetheless, its performance on hybrid CPUs does not meet the performance projection based on the CPU's hardware specification. Although previous research has optimized $llama.cpp$'s x86 CPU codes to achieve higher performance, it still falls short[16]. Both works utilize the traditional parallel method, resulting in high-performance cores waiting for low-performance cores.

To balance the workload among all cores, it is essential to understand the inference process of the LLM model, which consists of numerous kernels, each with a set of complex instructions. As the throughput of each instruction differs on different cores and the core's performance also varies from platform to platform, it is crucial to dynamically update the core performance and dispatch the kernel problem.

Although OpenMP declares that its parallel_for algorithm can dispatch the problem from core to core dynamically [13]. It uses a work-stealing algorithm [3] and range stealing to balance these partitions

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

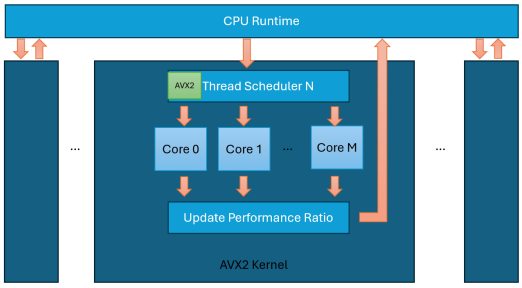

Figure 1: The dynamic LLM inference process

when workloads are unbalanced. But, the existing LLM inference framework, like $llama.cpp$, has precomputed and fixed partitions for each thread. It's not easy to change them to parallel_for style. On the other hand, splitting a matrix multiplication problem into small partitions is not regarded as beneficial. We propose to optimize only the task scheduling part of $llama.cpp$.

In this paper, we introduce a new dynamic parallel method for LLM inference within the framework of $llama.cpp$. We utilize Neural Speed [11] which highly optimizes x86 assembly codes based on the work of $llama.cpp$. Our primary contribution is the integration of our new parallel method into Neural Speed, allowing us to achieve more than 90% of memory bandwidth utilization on two hybrid CPUs during 4-bit LLM inference.

2 Approach

In this section, we introduce the two main components of our dynamic parallel method: a CPU runtime and a thread scheduler.

CPU runtime The CPU runtime records and manages the CPU status. Its thread pool binds each thread to a physical core and it tracks the execution time of each thread during executing kernels.

Thread scheduler The scheduler gets CPU status from the CPU runtime and divides kernel tasks into sub-tasks by each core's dynamic performance.

Figure 1 shows our dynamic process during LLM inference: the core performance table is updated after each layer's kernel execution. The engine will adjust the kernel workload among cores through the performance table and maximize the inference performance.

Assume there is a problem with problem size K, and there is a hybrid CPU with N cores. We define the performance ratio of i-th core as pr_i. pr_i may be different from each other in a hybrid CPU. The problem will be solved in parallel. Each core will solve a sub set of problem $ps_i = \theta_i\ K$, and $\theta_i \in (0, 1)\ \ sum\,(\theta_i) = 1$. The execution time of core i is $t_i = \frac{ps_i}{pr_i}$. The time to solve this problem T depends on the slowest core, $T = \max\,(t_i)$. Obviously, the best $\hat{\theta}$ to make the problem solved fastest is

$$\widehat{\theta_i} = \arg\min_{\theta_i} \left(\max \left(\frac{\theta_i\ K}{pr_i} \right) \right) = \frac{pr_i}{\sum pr_i} \tag{1}$$

However, the pr_i is determined by core frequency, CPU configuration, and even the system background program. We cannot get a static pr_i table before runtime.

2.1 CPU Runtime

CPU runtime is the runtime context of the CPU. It records the relative performance ratio of each core. We set all ratio values in the performance table at the initialization phase to 1: $pr_i = 1$. During runtime, the CPU updates the core performance ratio based on the Instruction Set Architecture (ISA), such as AVX2 and AVX-VNNI for Intel Ultra-125H. Because the P-cores (Performance) and E-cores (Efficiency) of Intel Ultra-125H have different instruction throughput and runtime frequency, it is important to note that different ISAs should have varying performance ratios. Saving performance

ratios for each kernel is preferable as a kernel typically utilizes more than one set of instructions. Our experiments have revealed that many kernels share the same performance ratio. As a result, we've designated a primary ISA for each kernel in our paper.

During LLM inference, the performance ratio will be dynamically updated to adjust the input problem size. If the current performance ratio of the i-th core is pr_i and the calculation time of i-th core is t_i in this calculation. The updated performance ratio $pr_i{}'$ is:

$$pr_i{}' = \frac{pr_i}{\sum_j t_i pr_j / t_j} \tag{2}$$

Considering the noise data, we use a filter to make our performance table more robust. The performance ratio is finally updated as this equation: $pr_i = \alpha * \ pr_i + (1 - \alpha) * pr_i{}'$ where α is the constant filter gain.

2.2 Thread scheduler

The Scheduler manages parallel problem dispatch for each computing kernel during the inference process, such as tensor copying and matrix multiplication. Its goal is to appropriately dispatch tasks among threads and ensure that all threads complete their sub-tasks simultaneously, even if the performance of each core varies. This approach maximizes CPU performance.

When a kernel involves parallel computations, a scheduler is created, and the ISA primarily used for these computations is specified in the code. The schedule queries the corresponding performance ratios from the CPU runtime based on the current ISA. The scheduler allocates tasks to each thread along a specific dimension according to the performance ratio. Each thread's sub-task proportion is equal to the ratio of its corresponding core's performance to the total performance. If the length of dimension is s, and the relative performance ratio of each core is pr_i, the sub-task of the i-th core s_i is

$$s_i{}' = \frac{pr_i}{\sum pr} \ s \tag{3}$$

Afterward, the scheduler activates the thread pool's interface to initiate the threads. Once all sub-tasks of the threads are finished, the scheduler records the execution time for each thread (core) from the thread pool and updates the CPU runtime's performance ratios.

The performance ratio will be distributed among different schedulers. This approach has the advantage of allowing the program to quickly adapt to the current computer conditions and maximize CPU performance, whether during program startup or when there are sudden changes in the system background.

3 Results

3.1 Experiment Setup

Hardware We evaluate the performance of two hybrid CPUs: Core-12900K and Ultra-125H. Both are hybrid CPUs with AVX-VNNI instruction.

Baselines 1. We compare the performance change of INT8 GEMM (General Matrix Multiply) between before and after our integration. We selected the AVX-VNNI GEMM micro kernel of Neural Speed to compute the GEMM problem in our test. We select OpenMP parallel method as a baseline. Please note that OpenMP here uses the balanced work dispatch algorithm. Each thread of OpenMP computes the same size of sub-matrix. We compare the latency of a GEMM problem case and a GEMV (General Matrix-Vector Multiplication) problem case. 2. We compare our model-level performance with Neural Speed and $llama.cpp$. We also select $llama.cpp$ as a baseline because its performance is known by more researchers.

LLM Model To demonstrate a stable result, we select llama2-7B in 4-bit weight-only quantization as it's widely used. The equivalent data type in $llama.cpp$ is Q4_0. It uses a group size of 32 for quantization, each group has 32 INT4 data and a FLOAT16 scale.

Metric Large Language Model (LLM) inference consists of two distinct phases - prefill phase which processes the input prompt and decode phase which generates output tokens autoregressively[1]. We will use the latency of the prefill phase to measure computing efficiency and the latency of the decode phase to measure memory bandwidth utilization.

3.2 Experiment Result

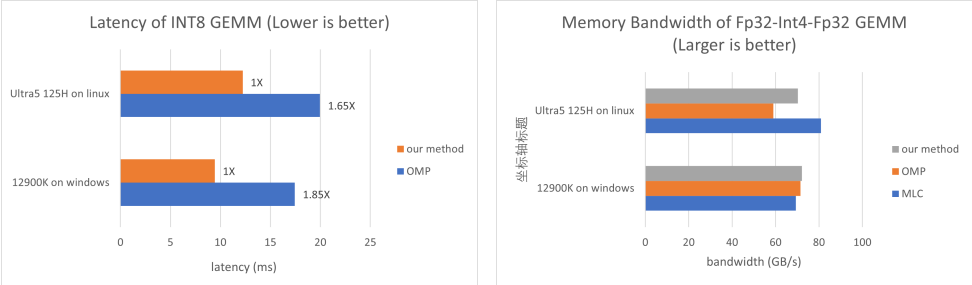

Figure 2: The latency and bandwidth of GEMM in different parallel methods

Firstly, we tested the performance of INT8 GEMM. The shape of GEMM is 1024x4096x4096, it usually occurs in the prefill phase. The data type of activation is unsigned INT8. The data type of weight is signed INT8. The data type of output is signed INT32. This GEMM problem is a compute-intensive workload. Our method demonstrates a 65% improvement in compute performance on Ultra-125H and an 85% enhancement on Core-12900K. This indicates that our method effectively utilizes the CPU's performance and enhances the collaboration of hybrid cores.

We also tested the performance of INT4 (Fp32-Int4-Fp32) GEMV. Unlike INT8 GEMM, this GEMV includes dynamic quantization for the FLOAT32 input tensor and dequantization for the FLOAT32 output tensor. This represents the complete computation of *llama.cpp* and Neural Speed. The shape of GEMV is 1x4096x4096. This shape is typical in the decode phase. In this test, we use MLC (Intel® Memory Latency Checker), a tool used to measure memory bandwidth, as a memory bandwidth reference. We convert the kernel's latency data to a memory bandwidth number, as shown in Figure 2. Our method shows a 19% bandwidth improvement on Ultra-125H. It achieves more than 90% memory bandwidth of the value MLC measured.

Secondly, we tested the performance of end-to-end LLM inference. We use two INT4 models of Neural Speed and *llama.cpp*. They are quantized with the same quantization parameters. It means that they have the same model size. The input prompt length is 1024. In this scenario, the prefill phase encountered a computational bottleneck, while the decode phase encountered a memory bandwidth bottleneck. This test shows improved computational efficiency and memory bandwidth utilization with our dynamic parallel method. In terms of the latency of the prefill phase,

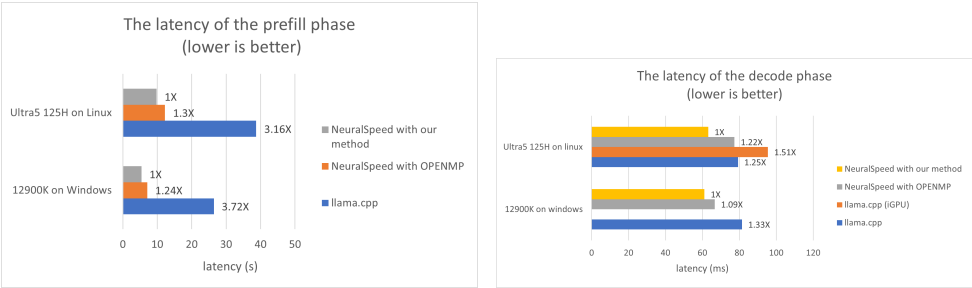

Figure 3: The latency of the prefill phase and the decode phase in Neural Speed (OpenMP and our method) and *llama.cpp*

our method represents an overall improvement of 20%-30% over the original OpenMP method in Neural Speed. One reason that the improvement is lower than the kernel's improvement is that we

only apply our method to GEMM kernels. Other kernels, like multi-head attention, do not benefit from our test. Our approach could significantly enhance the compute efficiency of two hybrid CPUs in compute-intensive workloads. As for the latency of the decode phase, our method is 9%-22% faster than the original OpenMP method in Neural Speed. The CPU decode speed is about 16 tokens/s.

We also examined how the performance ratios fluctuate during LLM inference. We documented the performance ratios of AVX-VNNI in Ultra-125H's P-cores as displayed in Figure 4. The constant filter gain α is 0.3.

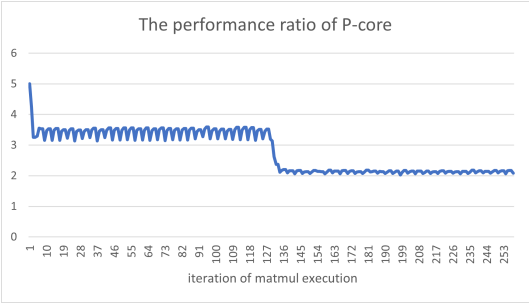

Figure 4: The performance ratio of one P-core in the prefill phase and the decode phase

There were two noticeable changes in the performance ratio, as depicted in the picture. The first change occurred after the program started. The performance ratio was initially set at 5, which was too high for this machine, but it quickly stabilized between 3 and 3.5. The second change occurred at the end of the prefill phase and the beginning of the decode phase. The inference of the prefill phase and the decode phase encountered different bottlenecks, resulting in different performance ratios. This test showed that our method can rapidly adapt to varying situations.

The data in Figure 4 explains the difference in speedup between the performance of the prefill phase and the decode phase. The larger the ratio, the greater the differences distributed between different cores in our method. Using the OpenMP method, the P-core would have to wait for the E-core for a longer time if the ratio is higher. Our method can dynamically find these ratios for different hybrid CPUs.

4 Summary and Future Work

Our approach enhances LLM inference performance on hybrid CPUs. The new parallel method replaces the OpenMP method of Neural Speed and shows promising performance improvements. With our help, Neural Speed achieves up to a 3.7x speedup compared to $llama.cpp$ and utilizes more than 90% of the memory bandwidth for INT4 GEMV. Under this memory bandwidth, the two CPUs in our test can generate tokens at about 16 tokens/s. The latest AIPC CPUs have additional compute units, such as GPU and NPU, allowing us to dynamically dispatch an LLM kernel between these units to utilize them simultaneously. Our next objective is to optimize the hybrid compute units to decrease the latency of the prefill phase on AIPC.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills, 2023.

[2] AMD. The Future of AI PCs Gets Even Better with AMD, 2024. https://www.amd.com/en/products/processors/consumer/ryzen-ai.html.

[3] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *J. ACM*, 46(5):720–748, sep 1999.

[4] Marc Brysbaert. How many words do we read per minute? a review and meta-analysis of reading rate. *Journal of Memory and Language*, 109:104047, 2019.

[5] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs, 2024.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs, 2023.

[7] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, 2023.

[8] Georgi Gerganov. llama.cpp, 2023. https://github.com/ggerganov/llama.cpp.

[9] Pujiang He, Shan Zhou, Wenhuan Huang, Changqing Li, Duyi Wang, Bin Guo, Chen Meng, Sheng Gui, Weifei Yu, and Yi Xie. Inference Performance Optimization for Large Language Models on CPUs, 2024.

[10] Intel. Neural Compressor, 2020. https://github.com/intel/neural-compressor.

[11] Intel. Neural Speed, 2023. https://github.com/intel/neural-speed.

[12] Intel. AIPC, 2024. https://www.intel.com/content/www/us/en/products/docs/processors/core-ultra/ai-pc.html.

[13] Microsoft. Parallel Algorithms, 2022. https://learn.microsoft.com/en-us/cpp/parallel/concrt/parallel-algorithms?view=msvc-170.

[14] Ditto PS, Jithin VG, and Adarsh MS. Inference Acceleration for Large Language Models on CPUs, 2024.

[15] Qualcomm. Qualcomm Unleashes Snapdragon X Elite: The AI Super-Charged Platform to Revolutionize the PC, 2023. https://www.qualcomm.com/news/releases/2023/10/qualcomm-unleashes-snapdragon-x-elite–the-ai-super-charged-plat.

[16] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. Efficient LLM Inference on CPUs, 2023.

[17] Xiaoxia Wu, Cheng Li, Reza Yazdani Aminabadi, Zhewei Yao, and Yuxiong He. Understanding INT4 Quantization for Transformer Models: Latency Speedup, Composability, and Failure Cases, 2023.